

Funktionsdokumentation — Aufbereitung

Stand 17.08.2026

Contents

prep/config.py	2
Modulkopf	2
Kommentarbloecke (19)	2
prep/s1_daten.py	5
Modulkopf	5
Kommentarbloecke (9)	6
Funktionen	7
prep/s2_datensaetze.py	9
Modulkopf	9
Kommentarbloecke (16)	9
Funktionen	11
prep/build.py	13
Modulkopf	13
Funktionen	13

prep/config.py

357 Zeilen · 0 Funktionen

Modulkopf

Konfiguration der Datenaufbereitung.

HIER STEHT, WAS IN DIE PARQUET-DATEIEN GESCHRIEBEN WIRD: Analysezeitraum, ausgeschlossene Stadtteile, Praediktoren, Zielgroessen, Klassen und die Zahl der Folds. Jede dieser Festlegungen bestimmt, welche Spalten die Datensaeetze haben oder wie sie belegt sind.

Was nur beim Rechnen gilt - Suchraeume, Tuning-Budget, Random State, Zahl der Wiederholungen - steht in modelle/config_modelle.py und beruehrt keine Datei auf der Platte.

Die Trennlinie ist also nicht "Daten gegen Modelle". N_FOLDS zum Beispiel steht hier, obwohl es nach Modellierung klingt: Es bestimmt die Spalte `fold` in beiden Datensaeetzen. Die Modellskripte lesen diese Festlegung, sie treffen sie nicht.

Bezug: docs/03_STAND.md, docs/02_ENTSCHEIDUNGEN.md

Kommentarbloecke (19)

Zeile 22

```
=====
1  PFADE
=====
```

Zeile 37

```
=====
2  DOWNLOADS (Schritt: prep/sl_daten.py)
=====
```

Alle Schalter stehen per Default auf False. `python prep/build.py` laeuft dann allein aus data/raw und braucht weder Internet noch API-Key.
Zum Neuladen einer Quelle den jeweiligen Schalter auf True setzen.

Zeile 69

Historische SFPD-Daten: Startdatum. Der Kriminalitaetsindex nutzt ein rollierendes 12-Monats-Fenster, das im VORMONAT endet; fuer den ersten Analysemonat 2015-01 werden daher die Delikte aus 2014 benoetigt.
Die Lag-Vorlaufmonate (2014) brauchen KEINEN Kriminalitaetsindex - sie liefern ausschliesslich Einsatzaehlungen fuer die Lags und werden nach der Lag-Bildung wieder entfernt.

Zeile 78

```
=====
3  JOINS (Schritt: prep/sl_daten.py)
=====
```

Publikationsverzoegerung der ACS-5-Jahres-Schaetzungen: Jahrgang y erscheint erst ca. Dezember y+1. Ein Modell, das im Jahr y schon den Jahrgang y nutzt, waere zum Prognosezeitpunkt nicht implementierbar (Decision Log #11).

Zeile 97

```
=====
4  ANALYSEZEITRAUM UND ANALYSEEINHEITEN
=====
```

=====

Hart fixiert, NICHT aus den Daten abgeleitet: Jeder Lauf liefert denselben Zeitraum, egal wie weit der letzte DataSF-Download reicht (Decision Log #18).

START 2015-01: frueheste Periode mit vollstaendigen ACS-Merkmalen unter Beruecksichtigung des Publikationsversatzes (Decision Log #5, #11).
ENDE 2025-12: letztes vollstaendiges Kalenderjahr (Decision Log #12).

Zeile 109

Lag-Vorlauf (Decision Log #23, 2026-07-27)

lag_12 fuer Januar 2015 braucht Januar 2014. Frueher fehlte dieser Monat im Panel, weshalb das erste Jahr je Stadtteil per dropna verlorenging und die Regression erst 2016-01 begann - waehrend die Klassifikation ab 2015-01 lief.

Die Lags brauchen ausschliesslich `anzahl\_einsaetze` aus der Vergangenheit, KEINE ACS-Merkmale. Der Grund fuer START = 2015 (Akademikerquote) betrifft nur die Praediktoren der Zielzeile, nicht deren Vergangenheitswerte. Einsatzzahlen liegen bis 2003 vor.

Ablauf: ab START-VORLAUF aggregieren -> Lags bilden -> auf START zuschneiden. Die Vorlaufmonate gehen ausschliesslich ueber shift() ein, nie als eigene Zeile. Ergebnis: 4.620 statt 4.200 Modellzeilen, beide Datensatze decken denselben Zeitraum ab.

Zeile 126

Stadtteile ohne nennenswerte Wohnbevoelkerung (Decision Log #19). Fuer ein bevoelkerungsbezogenes Risikomodell keine sinnvolle Analyseeinheit: jede Pro-Kopf-Groesse wird dort beliebig gross, weil der Nenner gegen null geht.

Golden Gate Park	45 Einwohner, Kriminalitaetsindex im Median 186
Lincoln Park	299 Einwohner
McLaren Park	507 Einwohner, zusaetzlich Census-Artefakt (Armutsquote 0,90)

Median aller uebrigen Stadtteile: 14.435 Einwohner.

Zeile 135

Ein Monat gilt als verdaechtig unvollstaendig, wenn seine stadtweite Einsatzzahl unter diesem Anteil des Median-Monats liegt. Nur Warnung, kein automatischer Filter - massgeblich bleibt ENDE.

Zeile 140

=====

5 MERKMALE DER REGRESSION

=====

Praediktoren gemaess Expose: soziooekonomisch, kriminalitaetsbezogen, baulich.

log_bevoelkerung statt roher Einwohnerzahl (Exposure, Decision Log #13): ohne diese Kontrolle sagt das Modell im Kern die Stadtteilgroesse vorher. Die Bevoelkerung ist die Groesse mit dem Vorzeichenwechsel - sie korreliert +0,20 mit der absoluten Einsatzzahl, aber -0,42 mit Einsaetzen je 1.000 Einwohner. (Die frueher hier genannte Armutsquote wechselt das Vorzeichen NICHT: +0,49 absolut, +0,46 pro Kopf. Korrektur vom 28.07.2026, docs/02_ENTSCHEIDUNGEN.md)

log_kriminalitaetsindex (Decision Log #17/#19): der Index ist ein Quotient, also multiplikativ und rechtsschief. Logarithmiert ist er symmetrisch um 0 (0 = Stadtdurchschnitt, +0,69 = doppelt so hoch). Fuer Baumverfahren ist die monotone Transformation wirkungsneutral, deshalb einheitlich fuer ALLE Modelle (Fairness-Regel).

Zeile 165

Rohwerte: keine Modellmerkmale, aber im Datensatz mitgefuehrt fuer den Offset des Poisson-GLM, die Raten-Sensitivitaet und die Deskription in Kap. 5.1. Bis Decision Log #45 (06.08.2026) hiess das hier "NegBin-Offset"; die Stufe-2-Baseline ist seither ein Poisson-GLM. Der Offset selbst ist unveraendert $\log(\text{gesamtbevoelkerung})$ - nur die Verteilungsannahme des Modells hat gewechselt, nicht die Rolle dieser Spalte.

Zeile 174

Saison: Kalendermonat als Sinus/Kosinus.
Der Monat als ZAHL 1-12 waere eine schlechte Kodierung - Dezember und Januar haetten den Abstand 11, obwohl sie benachbart sind, und ein linearer Koeffizient koennte ein U-foermiges Jahresmuster grundsaeztlich nicht abbilden. sin/cos legen die Monate auf ein Zifferblatt.

Zeile 181

Lags: Vergangenheitswerte der Zielgroesse, je Stadtteil. Sie bleiben im Datensatz, sind aber KEIN Modellmerkmal mehr (Decision Log #29).

Grund: Der Verfahrensvergleich laeuft seit dem 28.07.2026 ueber einen STADTTEIL-Split - trainiert wird auf 23 Stadtteilen je Fold, getestet auf unbekannten. Der Vormonatwert eines Teststadtteils waere dabei technisch verfuegbar, denn es ist seine eigene Vergangenheit. Genau dann erklart aber wieder seine Historie das Ergebnis statt seiner Struktur - und die Forschungsfrage bliebe unbeantwortet.

Wozu sie dann noch da sind: zur DESKRIPTION der zeitlichen Struktur in Kapitel 4 (Autokorrelation Lag 1 innerhalb eines Stadtteils). Es ist KEIN zweiter Analysestrang mit Zeitschnitt geplant - der waere ein zweiter Validierungsrahmen und verstiesse gegen R1 und R8 (praezisiert 04.08.2026, Decision Log #29). Sie formen den Datensatz auch nicht: Das dropna auf die Lags entfernt dank Vorlauf null Zeilen.
Leakage-sicher gebildet: strikt rueckwaertsgerichtet, `shift()` VOR `rolling()`.

Zeile 200

Ein Merkmalssatz - identisch fuer Ridge, Random Forest und XGBoost.

Bewusst NICHT enthalten: das rohe `jahr` und die Stadtteil-ID. Baumverfahren koennen nicht extrapolieren und ordnen unbekannte Werte dem letzten Blatt zu, waehrend Ridge linear weiterrechnet - das wuerde den Verfahrensvergleich verzerren. Eine Stadtteil-ID waere unter einem Stadtteil-Split ohnehin sinnlos: Der Teststadtteil ist im Training nie vorgekommen.

Zeile 211

=====

6 MERKMALE UND ZIELGROESSEN DER KLASSIFIKATION

=====

NFIRS-Codes sind hierarchisch; die fuehrende Ziffer bezeichnet die Serie. Zusammengefasst wird entlang der fachlichen Bedeutung, nicht nach Haeufigkeit (Decision Log #21).

100 Brand · 200 Ueberdruck/Explosion ohne Feuer · 300 Rettungsdienst
400 Gefahrenlage · 500 Serviceeinsatz · 600 Good Intent · 700 Fehlalarm
800 Naturereignis · 900 Sonstige

Zeile 234

Zielgroessen der Klassifikation: die ZUSAMMENSETZUNG der Einsatzlast je Stadtteil und Monat, nicht die Art des einzelnen Einsatzes (Decision Log #29).

Warum der Wechsel: Innerhalb eines Stadtteil-Monats tragen ALLE Einsaetze identische Strukturmerkmale. 350.481 Einzeleinsaetze enthielten nur 4.619

verschiedene Profile; ein perfektes Modell auf den Strukturmerkmalen haette 49,9 % Treffer erreicht gegenueber 48,2 % fuer blosses Raten. Auf Stadtteil-ebene ist dieselbe Frage dagegen beantwortbar: Der Fehlalarm-Anteil laesst sich fuer einen unbekannten Stadtteil mit R2 0,66 vorhersagen.

Zeile 255

Ergebnisvariablen - duerfen NIEMALS Merkmal sein. Sie stehen erst nach dem Einsatz fest oder sind eine Folge der Einsatzart; ihre Verwendung waere Leakage im engeren Sinn (Decision Log #20).

Zeile 266

```
=====
7  VALIDIERUNG (Schritt: prep/s2_datensaetze.py, Teil A)
=====

STADTTEIL-SPLIT (Decision Log #29). Die Forschungsfrage lautet: Laesst sich
aus Strukturmerkmalen vorhersagen, wie viele und welche Einsaetze ein
Stadtteil hat? Diese Frage prueft man, indem man einen Stadtteil komplett
zurueckhaelt - nicht, indem man die Zeitachse schneidet. Bei einem Zeitschnitt
steht jeder Stadtteil in Training UND Test; das Modell kennt sein Niveau
bereits und die Strukturmerkmale muessen nichts erklaren.

29 Stadtteile -> 5 Folds (6/6/6/6/5) 6 Stadtteile -> Hold-out
jeder Stadtteil ist genau einmal Testfall, nie zugleich Trainingsfall

Zuteilung stratifiziert nach Bevoelkerung: Die Stadtteile werden nach
Einwohnerzahl sortiert und reihum auf die Gruppen verteilt. Damit deckt jede
Gruppe die gesamte Groessenspanne ab - sonst laege im Test zufaellig nur
Downtown oder nur Seacliff. Stratifiziert wird nach einem PRAEDIKTOR, nicht
nach der Zielgroesse; sonst floesse Testinformation in die Gruppenbildung ein.
Die Stadtteile werden reihum auf N_FOLDS + 1 Gruppen verteilt. Gruppe 0 ist
das Hold-out, die Gruppen 1..N_FOLDS sind die Folds. Bei 35 Stadtteilen
ergibt das 6 Hold-out-Stadtteile und Folds der Groesse 6, 6, 6, 6, 5.
```

Zeile 289

```
=====
8  SPALTENNAMEN  englisch -> deutsch
=====

Die Rohquellen (DataSF, Census) liefern englische Namen; ab dem Ende von
prep/sl_daten.py heisst im Projekt alles deutsch. Dieses Mapping ist die
einzige Stelle, an der der Wechsel passiert. Steht hier unten, weil man es
beim Arbeiten praktisch nie anfasst - im Gegensatz zu allem darueber.
```

Zeile 333

— SFPD Kriminalitaet —
Relativer Index je Stadtteil x Monat (Location Quotient gegen den
Stadtdurchschnitt desselben Monats, rollierendes 12-Monats-Fenster endend
im Vormonat). Ersetzt die frueheren statischen Anteile.

prep/sl_daten.py

595 Zeilen · 14 Funktionen

Modulkopf

Schritt 1: Rohdaten beschaffen und auf Einsatzebene zusammenfuehren.

Eingang: DataSF- und Census-APIs (nur wenn DOWNLOAD_* in config.py True ist)
data/raw/*

Ausgang: data/processed/einsaetze.parquet (ein Einsatz je Zeile, ~720.000)

Vier Quellen: ACS (soziooekonomisch, je Stadtteil x Jahrgang, mit Publikationsversatz), Crime (relativer Index je Stadtteil x Monat), Land Use (baulich, Snapshot 2020) und die Neighborhood-Geometrie fuer beide Spatial Joins. Die Ebene bleibt der EINZELEINSATZ - aggregiert wird erst in s2_datensaetze.py.

Ausfuehren:

```
python prep/s1_daten.py          # Download (soweit aktiviert) + Join
python prep/s1_daten.py join     # nur Join
```

Kommentarblöcke (9)

Zeile 65

```
=====
TEIL A  DOWNLOAD
=====
```

Gesteuert allein ueber die DOWNLOAD_*-Schalter in config.py. Stehen alle auf False (Default), arbeitet die Aufbereitung aus data/raw und braucht weder Internet noch API-Key.

```
=====
Deklarative Quellenliste. Jede DataSF-Quelle unterscheidet sich nur in
Ressourcen-ID, Feldauswahl, Filter und Nachbearbeitung der Spaltentypen -
deshalb eine Tabelle und eine Schleife statt sechs fast gleicher Funktionen.
```

Zeile 223

```
=====
TEIL B  SFFD - Einsatzdaten
=====
```

Zeile 247

```
=====
TEIL C  ACS - soziooekonomische Merkmale
=====
```

Zeile 301

```
=====
TEIL D  GEOMETRIE
=====
```

Zeile 313

```
=====
TEIL E  CRIME - relativer Kriminalitaetsindex je Stadtteil x Monat
=====
```

Zeile 436

```
=====
TEIL F  LAND USE - bauliche Merkmale
=====
```

Zeile 490

```
=====
TEIL G  QUOTEN
=====
```

Zeile 521

=====

Ablauf

=====

Zeile 553

Kein statischer Fallback: Ein statischer Crime-Join wuerde Delikte aus dem Testzeitraum in die Trainingsmerkmale tragen (Leakage) und jede Zeitvarianz beseitigen.

Funktionen

_get(url, params)

Zeile 112 · 5 Zeilen

kein Docstring

lade_datasf(name, limit)

Zeile 119 · 41 Zeilen

Eine DataSF-Quelle vollstaendig holen und die Spaltentypen setzen.

lade_acs(year)

Zeile 162 · 21 Zeilen

ACS 5-Year Estimates auf Tract-Ebene fuer San Francisco County.

Eigene Funktion, weil die Census-API ein anderes Format liefert als DataSF: Kopfzeile plus Datenzeilen als verschachtelte Liste.

run_download()

Zeile 185 · 36 Zeilen

kein Docstring

prepare_sffd(df)

Zeile 226 · 19 Zeilen

Dedup, Antwortzeit, Zeit-Features, Stadtteilnamen normalisieren.

acs_je_neighborhood(acs, crosswalk)

Zeile 250 · 15 Zeilen

Tract -> Neighborhood. Mediane bevoelkerungsgewichtet, Zaehler summiert.

acs_snapshot(jahr, acs_years)

Zeile 267 · 14 Zeilen

Letzter zum Prognosezeitpunkt TATSAECHLICH PUBLIZIERTER ACS-Jahrgang.

Bedingung: `acs_jahr <= Einsatzjahr - ACS_PUBLIKATIONS_LAG`. Zwei Stufen der Absicherung: "letzter verfuegbarer" statt "zeitlich naechster" Snapshot (Decision Log #4) und zusaetzlich die reale Publikationsverzoegerung von ~1 Jahr (#11) - ohne sie haette ein Einsatz aus 2023 den Jahrgang 2023

bekommen, der erst Ende 2024 erschienen ist.

Vor dem ersten Snapshot gibt es keinen vergangenen Jahrgang; Rueckgriff auf den aeltesten als dokumentierte Limitation - die Hauptanalyse beginnt 2015.

join_acs(sfdd, nb_per_year)

Zeile 283 · 16 Zeilen

Jeder Einsatz bekommt den zu seinem Jahr passenden ACS-Snapshot.

neighborhoods_gdf()

Zeile 304 · 7 Zeilen

Neighborhood-Polygone. Beide Spatial Joins nutzen dieselbe Geometrie, damit sich Kriminalitaets- und Baumerkmale auf identische Flaechen beziehen.

crime_monatlich()

Zeile 316 · 51 Zeilen

Deliktzahlen je Neighborhood und Monat aus beiden SFPD-Quellen.

Modern (e3si-785i, ab 2018-01) ist bereits voraggregiert; da der Index ALLE Straftaten zaehlt, werden die Kategorien summiert - eine Harmonisierung der Kategorienschemata eruebrigt sich. Historisch (tmnf-yvry, bis 2017) hat keine Stadtteilstalte, daher Spatial Join der Koordinaten.

kriminalitaetsindex(nb_per_year)

Zeile 369 · 65 Zeilen

Relativer Kriminalitaetsindex je Neighborhood und Monat.

Definition (Location Quotient der Kriminalitaetsbelastung):

$$\begin{aligned} \text{rate}(i,t) &= \text{Delikte}(i, \text{Fenster endend in } t-1) / \text{Einwohner}(i) \\ \text{rate}(\text{Stadt},t) &= \text{Delikte}(\text{Stadt}, \text{gleiches Fenster}) / \text{Einwohner}(\text{Stadt}) \\ \text{index}(i,t) &= \text{rate}(i,t) / \text{rate}(\text{Stadt},t) \end{aligned}$$

Lesart: 1,0 = Belastung wie im Stadtdurchschnitt desselben Monats.

Warum relativ statt absolut? Der SFPD-Systemwechsel im Mai 2018 veraendert das stadtweite Niveau. Ein multiplikativer Niveausprung wirkt auf Zaehler und Nenner gleich und kuerzt sich heraus. Verbleibende Limitation: Eine Verschiebung in der ZUSAMMENSETZUNG der erfassten Delikte, die einzelne Stadtteile staerker trifft, kuerzt sich nicht heraus (Kap. 6.3).

Kein Leakage: Das Fenster endet strikt im Vormonat.

`crime\_rate\_raw` (Delikte je 1.000 Ew.) ist NUR deskriptiv fuer Kapitel 5.1, kein Modellmerkmal - sie enthaelt den Bruch von 2018.

land_use_je_neighborhood()

Zeile 439 · 49 Zeilen

Parzellen-Centroid -> Neighborhood, dann Aggregation je Neighborhood.

Statisch (Snapshot 2020, einziger verfuegbarer Jahrgang).

berechne_quoten(df)

Zeile 505 · 14 Zeilen

```
Anteilswerte in [0,1]; Nenner <= 0 ergibt NaN statt Division durch Null.  
  
Kriminalitaet taucht hier NICHT auf: Sie geht als relativer Index je  
Stadtteil x Monat ein (Decision Log #17), nicht als Anteil.
```

run_join()

Zeile 524 · 66 Zeilen

kein Docstring

prep/s2_datensaetze.py

445 Zeilen · 9 Funktionen

Modulkopf

Schritt 2: die beiden finalen Datensaetze samt Validierungsrahmen.

```
Eingang: data/processed/einsaetze.parquet      (ein Einsatz je Zeile)  
Ausgang: data/processed/regression.parquet      Stadtteil x Monat, Menge  
         data/processed/klassifikation.parquet  Stadtteil x Monat, Struktur
```

BEIDE liegen auf derselben Analyseeinheit - Stadtteil x Monat. Die eine misst die MENGE der Einsatzzlast, die andere ihre ZUSAMMENSETZUNG. Damit laufen beide Teile der Arbeit durch denselben Rahmen: gleiche Zeilen, gleiche Merkmale, gleiche Folds (Gutachten R1, Decision Log #29).

Der Validierungsrahmen steht ebenfalls hier, weil die Aufteilung als SPALTEN in die Dateien geschrieben wird. "Alle Verfahren sehen identische Folds" ist damit eine Zusage ueber den DATENSATZ, nicht ueber die Algorithmen - sie haengt nicht davon ab, dass jedes Modellskript die richtige Funktion aufruft.

```
TEIL A  Stadtteil-Split: Folds und Hold-out  
TEIL B  Menge          Aggregation, Exposure, Rate, Saison, Lags  
TEIL C  Struktur       Anteile der vier NFIRS-Gruppen
```

Steckbrief und Kennzahlen der erzeugten Dateien: docs/03_STAND.md

Ausfuehren:

```
python prep/s2_datensaetze.py      # beide Datensaetze bauen  
python prep/s2_datensaetze.py splits # nur die Aufteilung anzeigen
```

Kommentarbloecke (16)

Zeile 60

```
=====
```

```
TEIL A  STADTTEIL-SPLIT
```

```
=====
```

Geprueft wird, indem ganze Stadtteile zurueckgehalten werden: 6 ins Hold-out, die uebrigen 29 auf 5 Folds (6/6/6/6/5), jeder genau einmal Testfall. Ein Zeitschnitt wuerde die Forschungsfrage nicht pruefen - dort steht jeder Stadtteil in Training UND Test (Decision Log #29).

```
=====
```

Zeile 140

```
=====
Von beiden Datensätzen genutzt
=====
```

Zeile 175

```
=====
TEIL B  REGRESSION
=====
```

Zeile 219

Nur VORWAERTS füllen. KEIN bfill: Rueckwaertsfüllen würde fehlende Werte (z. B. akademikerquote vor ACS 2014) still mit ZUKUNFTSWERTEN imputieren - Leakage (Decision Log #10). Echte NaN bleiben sichtbar.

Zeile 229

Kriminalitaetsindex logarithmieren (#17/#19): 0 = Stadtdurchschnitt. Nullwerte wurden -inf erzeugen und werden zu NaN, damit sie sichtbar bleiben statt still zum Extremwert zu werden.

Zeile 260

LAGS - Strikt rueckwaertsgerichtet und je Stadtteil gebildet, nie ueber Stadtteilgrenzen hinweg. Beim gleitenden Mittel steht `shift(1)` VOR `rolling(3)`: der Wert fuer Monat t verwendet t-1, t-2, t-3, nie t selbst.

Zeile 269

Vorlauf abschneiden. Erst danach greift die NaN-Pruefung des balancierten Panels - die Vorlaufmonate haben absichtlich keine Strukturmerkmale (der Kriminalitaetsindex beginnt erst 2015-01) und duerfen die Stadtteilauswahl nicht beeinflussen.

Zeile 279

Balanciertes Panel (Decision Log #15): Stadtteile ohne durchgaengige ACS-Abdeckung fliegen GANZ raus. Zeilenweises dropna erzeugte sonst ein unbalanciertes Panel - ein Stadtteil tritt mitten in der Zeitreihe hinzu.

Zeile 290

Sicherheitsnetz bei zu kurzem Vorlauf: Anlaufmonate ohne lag_12 entfallen fuer ALLE Verfahren gleichermassen, sonst liefen sie auf unterschiedlichen Zeilen (Fairness-Regel).

Zeile 298

Zweite Zielgroesse: Einsaetze je 1.000 Einwohner. Fuer den Vergleich zwischen unterschiedlich grossen Stadtteilen ist die Rate die aussagekraeftigere Groesse - die absolute Zahl bildet vor allem die Einwohnerzahl ab (Decision Log #29).

Zeile 304

REPRODUZIERBARKEITSVERTRAG - diese Sortierung darf nicht veraendert werden: Random Forest und XGBoost ziehen ihre Bootstrap- bzw. Subsample-Stichproben ueber Zeilenpositionen. Eine andere Reihenfolge liefert trotz identischem random_state leicht andere Baeume (empirisch 17,2587 statt 17,2974 RMSE in Fold 1). Ridge ist reihenfolgeinvariant.

Zeile 315

```
=====
TEIL C  STRUKTUR DER EINSATZLAST
=====
```

Zeile 364

Zielgroesse der Klassifikation: die haeufigste Einsatzart des Monats.
Eine echte Klasse, kein gesetzter Schwellwert - argmax ueber die vier NFIRS-Gruppen. Damit entfaellt die Begrueudungslast, die eine kuenstliche Einteilung einer Zaehlgroesse mit sich braechte (Altman & Royston 2006).

Zeile 395

```
=====
Ablauf
=====
```

Zeile 408

Fold-Zuteilung EINMAL fuer beide Datensaeetze, stratifiziert nach der seltensten Klasse und der Bevoelkerung. Sie muss identisch sein, sonst waeren die Ergebnisse der beiden Straenge nicht vergleichbar.

Zeile 423

Einzige Kennzahl, die hier gedruckt wird: die Brand-Testfaelle je Fold.
Sie ist der Grund fuer die doppelte Stratifizierung (#30) und der einzige Wert, der beim Lauf tatsaechlich kontrolliert werden muss.
Alles Weitere steht in docs/03_STAND.md und wird von den Tests geprueft.

Funktionen

ergaenze_aufteilung(daten, versatz, selten)

Zeile 68 · 35 Zeilen

Schreibt `fold` (0..N_FOLDS) und `ist\_holdout` in den Datensatz.

Die Stadtteile werden reihum auf N_FOLDS + 1 Gruppen verteilt; Gruppe 0 ist das Hold-out. Wer wohin kommt, haengt allein von der Sortierreihenfolge ab, und die stratifiziert doppelt (Decision Log #30):

1. `selten` nach der Zahl brand-dominierten Monate. Ohne dieses Kriterium hatte ein Fold regelmaessig KEINEN Brand-Testfall - Macro-F1 mittelt dann ueber eine im Test nicht vorhandene Klasse.
2. `bev` nach Bevoelkerung bei Gleichstand, damit kein Fold nur aus Grossstadtteilen besteht. Sonst waere die Fold-Streuung ein Groesseneffekt statt eines Modellunterschieds.

Kein Leakage: Das Modell bekommt keine zusaetzliche Information, es wird nur festgelegt, welche Stadtteile gemeinsam getestet werden - wie bei `StratifiedGroupKFold`.

`versatz` verschiebt den Startpunkt der Austeilung, fuer WIEDERHOLTE Splits. Bei 29 Entwicklungsstadtteilen schwankt ein einzelner Fold stark; erst ueber Wiederholungen gemittelt ist die Schaetzung stabil.

fold_masken(daten, k)

Zeile 105 · 12 Zeilen

Trainings- und Testmaske des Folds k - allein aus den Spalten der Datei.

Test = die Stadtteile dieses Folds, mit allen ihren Monaten.
Train = alle uebrigen Entwicklungs-Stadtteile, ohne das Hold-out.
Kein Stadtteil ist je zugleich Trainings- und Testfall.

beschreibe_splits(daten)

Zeile 119 · 19 Zeilen

Menschenlesbare Zusammenfassung der Aufteilung (fuer Kap. 5.2/5.4).

Welcher Stadtteil in welchem Fold getestet wird, und dass jeder Fold den vollen Zeitraum abdeckt - das unterscheidet den Stadtteil-Split vom frueher verwendeten Zeitschnitt.

_monat_minus(jahr_monat, monate)

Zeile 143 · 5 Zeilen

Verschiebt einen jahr_monat-Schluessel um n Monate zurueck.

_setze_datentypen(d, merkmale)

Zeile 150 · 23 Zeilen

Vereinheitlicht die Datentypen auf modelltaugliche NumPy-Typen.

Merkmale float64 · Schluessel, Zaehlgroessen und Steuerspalten int64 · stadtteil str.

Notwendig, weil EINE nullable Int64-Spalte im Merkmalssatz genuegt, damit `X.to\_numpy()` ein object-Array liefert - sklearn faengt das still ab, XGBoost lehnt es ab (Decision Log #24).

aggregiere(von, bis, mit_parkgebieten, verbose)

Zeile 178 · 64 Zeilen

Einsatz-Ebene -> Stadtteil x Monat, vollstaendiges Raster.

`von`/`bis` sind jahr_monat-Schluessel INKLUSIVE Lag-Vorlauf.

baue_regression(vorlauf, verbose)

Zeile 244 · 69 Zeilen

Der vollstaendige Regressionsdatensatz.

LAG-VORLAUF (Decision Log #23): Aggregiert wird ab START minus `vorlauf` Monaten, damit lag_12 schon fuer den ersten Analysemonat definiert ist. Danach Zuschnitt auf START - die Vorlaufmonate gehen ausschliesslich ueber shift() ein, nie als eigene Zeile.

baue_klassifikation(regression, verbose)

Zeile 318 · 75 Zeilen

Anteile der vier NFIRS-Gruppen je Stadtteil und Monat.

Zielgroesse ist die ZUSAMMENSETZUNG der Einsatzlast, nicht die Art des einzelnen Einsatzes (Decision Log #29). Innerhalb eines Stadtteil-Monats tragen alle Einsaetze identische Strukturmerkmale; auf Einzeleinsatz-Ebene war deshalb nichts zu holen - ein perfektes Modell haette 49,9 % Treffer erreicht gegenueber 48,2 % fuer blosses Raten. Auf dieser Ebene ist die Frage beantwortbar.

Zeilen, Zeitraum, Stadtteile, Merkmale und Folds werden dem Regressionsdatensatz ENTNOMMEN. Beide Teile der Arbeit beruhen damit zwingend auf demselben Datenbestand und derselben Aufteilung.

run(verbose)

Zeile 398 · 34 Zeilen

Beide finalen Datensätze bauen, Folds eintragen und schreiben.

Die Fold-Zuteilung erfolgt EINMAL und wird auf beide Datensätze angewandt - nur so sehen Menge und Struktur dieselben Stadtteile im Test (Fairness-Regel).

prep/build.py

92 Zeilen · 3 Funktionen

Modulkopf

DER EINE BEFEHL. Erzeugt aus den Rohdaten die beiden finalen Datensätze.

```
python prep/build.py
```

Ablauf:

```
1 s1_daten.py          -> data/raw/* (nur was in config.py auf True steht)
                           -> data/processed/einsaetze.parquet Zwischenstand
2 s2_datensaetze.py    -> data/processed/regression.parquet FINAL
                           -> data/processed/klassifikation.parquet FINAL
```

Danach ist die Aufbereitung fertig. Die beiden FINAL markierten Dateien sind modellfertig: identische Zeilen, Merkmale und Folds fuer alle Verfahren.

Dieser Ordner erzeugt DATEN und sonst nichts. Die Messlatte (Baselines) und die Eignungspruefung liegen in vorpruefung/, der Verfahrensvergleich in modelle/.

Argumente (optional):

```
python prep/build.py daten      wie ohne Argument
python prep/build.py tests      anschliessend die Pruefungen laufen lassen
```

Downloads werden ueber die DOWNLOAD_*-Schalter in config.py gesteuert. Stehen sie auf False (Default), arbeitet der Befehl allein aus data/raw und braucht weder Internet noch API-Key.

Funktionen

schritt(nummer, titel)

Zeile 43 · 3 Zeilen

kein Docstring

uebersicht()

Zeile 48 · 16 Zeilen

kein Docstring

main()

Zeile 66 · 23 Zeilen

kein Docstring